Political Promise Fact-Checker

An NLP-Based Historical Analysis & Outcome Prediction System

This document provides a complete technical walkthrough of the project — from raw data to final prediction — written for faculty presentation. Every concept is explained from first principles.

Subject	Natural Language Processing (NLP)
Focus Party	Bharatiya Janata Party (BJP)
Data Coverage	2014 Manifesto + Review | 2019 Manifesto + Review | 2024 Manifesto
Core NLP Model	all-MiniLM-L6-v2 (Sentence Transformers)
LLM Reviewer	LLaMA 3.3 70B via Groq API (secondary layer)
API Framework	FastAPI (Python)
Output	Verdict: Likely Fulfilled / Partially Fulfilled / Unlikely / Indeterminate

1. What Does This Project Do?

The Political Promise Fact-Checker is a system that takes any political promise (text) as input and predicts **how likely it is to be fulfilled**, based on 10 years of BJP's actual governance history.

Simple analogy: Imagine a student who always promises to study but never does. If they promise again, you'd say 'Unlikely to be fulfilled' because of their track record. This system does the same — but for political promises, using NLP.

The system answers questions like:

- Has BJP made this kind of promise before?
- When they made it before, did they actually deliver?
- Based on that history, what is the likely outcome this time?

2. Data Sources — What Raw Data Do We Have?

All data is stored in the **data/raw/** folder. We use three types of sources:

File / Folder	Type	What It Contains	Used For
Manifesto_English.pdf	PDF	BJP 2014 Election Manifesto — all promises made before 2014 elections	Source of 2014 promises
2014 BJP Manifesto Review.pdf	PDF	Independent review of BJP's 2014-2019 term — what was done and what was not	Labels for 2014 promises
BJP-Election-english-2019.pdf	PDF	BJP 2019 Election Manifesto — all promises made before 2019 elections	Source of 2019 promises
data/raw/bjp_2019/ (9 CSV files)	CSV	Sector-wise report card of BJP 2019-2024 term. Sectors: Agriculture, Economy, Health, Infra, S	Labels for 2019 promises
BJP-Election-english-2024.pdf	PDF	BJP 2024 Election Manifesto — the promises we want to predict outcomes for	Target for prediction

The 2019 CSV report cards look like this (one row = one promise):

PROMISE	STATUS AND COMMENTS	SECTOR
Ensure LPG gas cylinder connection to all poor households	Completed under Ujjwala Yojana — 8 crore connections given	INFRA
Double the Farmers' Income (2016-2022)	Target not met. Farm income grew but did not double by 2022	AGRICULTURE

3. Full Data Processing Pipeline

The system works in a two-phase pipeline:

Phase 1 — Build the Gold Database (run once, offline)

This is done by running **rebuild_gold.py**. It creates a knowledge base of 524 historical promise-outcome pairs.

Step	What Happens	Code File
Step 1	PDF Parser reads the 2014 Review PDF page-by-page, extracts tables row-by-row. Each row = one promise with one remark	pdf_parser.py — extract_review_table()
Step 2	CSV Reader reads all 9 sector CSVs from bjp_2019/ folder. Merges them into one combined dataframe with a 'sector' colu	pdf_parser.py — load_2019_folder()
Step 3	Semantic Labeler converts each 'remark' text into a numeric label: 2 = Fulfilled, 1 = Partial, 0 = Unfulfilled. Uses NLP similarit	nlp_engine.py — process_labels()
Step 4	Both years are merged with a 'year' column (2014/2019) and saved as gold_database.csv — 524 total records	rebuild_gold.py

Phase 2 — Real-Time Prediction (runs on every API request)

This is done by **main.py** (FastAPI server). When a user sends a promise text via POST /predict:

Step	What Happens	Technical Term
Step 1	The input promise text is converted into a 384-dimensional vector (a list of 384 numbers representing meaning)	Text Embedding / Sentence Embedding
Step 2	This vector is compared against all 524 vectors in the Gold Database using Cosine Similarity	Semantic Search
Step 3	Top 3 most similar historical records are retrieved with their similarity scores (e.g., 0.82, 0.71, 1.0)	Top-K Retrieval
Step 4	The label of the best match is used as the base prediction. Confidence thresholds are applied (≥0.70, ≥0.60, ≥0.50)	Threshold-based Classification
Step 5 (Optional)	The semantic output is sent to LLaMA 3.3 70B via Groq API for a political secretary review	LLM Secondary Review
Step 6	Final verdict is assembled and returned as JSON	API Response

4. The Core NLP Model — all-MiniLM-L6-v2

This is the heart of the system. It is a **Sentence Transformer** model — a type of BERT-based neural network pre-trained on 1 billion sentence pairs.

What is a Sentence Transformer?

Normal word-level models (like basic BERT) understand individual words. A Sentence Transformer understands the **meaning of an entire sentence**. It converts any sentence into a fixed-size vector of 384 numbers — called an **embedding**.

Example: "We will build highways across India" → [0.23, -0.41, 0.89, ... 384 numbers] "Road infrastructure development across the country" → [0.21, -0.38, 0.91, ... 384 numbers] These two vectors are very CLOSE → High similarity → Same topic

Why this specific model?

Property	Value	Why It Matters
Model Name	sentence-transformers/all-MiniLM-L6-v2	Open-source, no API key needed
Architecture	6-layer MiniLM (distilled BERT)	Fast enough to run on CPU
Embedding Dimension	384	Compact but highly accurate
Training Data	1 Billion sentence pairs	Understands diverse English including political language
Model Size	~80 MB	Downloads once, runs offline forever
Inference Speed	~25ms per sentence (CPU)	Real-time API response

What is Cosine Similarity?

After encoding two sentences into vectors, we measure how 'close' they are using Cosine Similarity. The formula is:

```
Cosine Similarity = cos(θ) = (A · B) / (||A|| × ||B||)
```

The result is a number between 0 and 1:

Score	Meaning	Example
0.90 – 1.00	Near-identical meaning (exact match)	'Build Ram Temple' vs 'Construct Ram Mandir in Ayodhya'
0.70 – 0.89	Same topic, very similar promise	'LPG to poor families' vs 'Gas connections to rural households'
0.50 – 0.69	Related topic, different angle	'Financial inclusion' vs 'Zero-balance bank accounts'
Below 0.50	Unrelated or no good match	No historical precedent found

5. How Are Promises Labeled? (Semantic Labeling)

The remark column in each CSV says things like *'Completed under Ujjwala Yojana'* or *'Target not met, still pending'*. We need to convert these free-text remarks into 3 numeric labels:

Label	Numeric Value	Meaning	Example Remark Text
Highly Likely / Fulfilled	2	Promise was completed or has strong momentum	'Completed and operational', 'Successfully implemente
Partial	1	Some progress was made but target not fully achieved	'Work in progress', 'Partially implemented', 'Ongoing'
Unlikely	0	No progress, cancelled, or repeatedly delayed	'Nil progress', 'Stalled', 'Not achieved'

How Does Semantic Labeling Work?

Instead of using keyword matching (which is fragile), we use the NLP model itself to assign labels. We define 6 'anchor sentences' — 2 per label — and measure which anchor each remark is most similar to:

Label	Anchor Sentences Used
2 (Fulfilled)	"Project completed and operational." | "Successfully implemented and achieved."
1 (Partial)	"Work in progress and ongoing." | "Partial implementation underway."
0 (Unlikely)	"No progress made, stalled." | "Cancelled or likely to be unfulfilled."

The remark gets the label of whichever anchor it's closest to in vector space. This is more robust than keyword matching because it understands **meaning**, not just words.

6. The Gold Database

The Gold Database (**data/processed/gold_database.csv**) is the core knowledge base. It has 524 rows — each row is one historical promise with its actual outcome label.

original_text	label	year	sector
Ensure LPG gas cylinder connection to all poor rural households	2	2019	INFRA
Double the Farmers' Income (2016-2022)	0	2019	AGRICULTURE
BJP reiterates its stand on the Article 370...	1	2014	—
We will launch Diamond Quadrilateral project of High Speed Train network	2	2014	—

Total records: **524** — split roughly as:

- **~185 records** labeled 0 (Unlikely) — promises that failed
- **~30 records** labeled 1 (Partial) — partial delivery
- **~309 records** labeled 2 (Highly Likely / Fulfilled) — completed

Note: The label distribution is skewed toward '2' because BJP's review documents tend to report successes. The LLM layer provides a critical second opinion to compensate for this bias.

7. Confidence Tiers & Thresholds

Once the best historical match is found, the similarity score determines the confidence tier of the prediction. These thresholds were set based on empirical testing:

Similarity Score	Confidence Tier	What It Means	Example
≥ 0.70	High Confidence	Strong match — prediction is reliable	Article 370 (0.96), Diamond Quadrilateral (0.947)
0.60 – 0.69	Moderate Confidence	Good match — prediction is reasonable	Article 370 2014 remark (0.61)
0.50 – 0.59	Low Confidence	Weak match — treat with caution	Financial inclusion promises (~0.55)
< 0.50	Indeterminate	No relevant history found	Very novel or vague promises

Why 0.70 as High Confidence threshold?

In practice, political promises use varied language. A 0.70 similarity in sentence embeddings represents semantically near-identical content — it reliably finds the 'same' promise made in a previous term. Below 0.70, there's enough linguistic distance that the match could be coincidental.

8. LLM Integration — Groq + LLaMA 3.3 70B

The LLM (Large Language Model) is a **secondary reviewer**, not the primary system. The NLP semantic output is the authoritative answer. The LLM adds political and real-world context on top.

Why Is the LLM Secondary?

This is a key architectural decision. The NLP model is fully deterministic, auditable, and based on labeled historical data from actual BJP review reports. The LLM is probabilistic and based on general training data. For an academic NLP project, the primary output must come from the NLP pipeline.

What Does the LLM Do?

LLM Role	Description
Independent Reviewer	Analyzes the semantic output and gives its own verdict based on real-world political knowledge (e.g., kn
Discrepancy Detector	When LLM disagrees with semantic output, a discrepancy flag is raised — useful for identifying mislabel
Context Provider	Adds specific scheme names, policy names, budget allocations, governance context that the NLP mode
Repeat Promise Detector	Flags if a 2024 promise is identical to a past manifesto promise (repeat_promise: true)

Model Used: LLaMA 3.3 70B via Groq

We use Groq's free API tier with the LLaMA 3.3 70B model. Key parameters:

Parameter	Value	Reason
model	llama-3.3-70b-versatile	70B parameters — best reasoning available for free
temperature	0.1	Near-deterministic — same promise gives same verdict every time
max_tokens	600	Enough for structured JSON response with full reasoning
response_format	json_object	Forces valid parseable JSON — prevents hallucinated text

9. Real Example Walkthrough — Article 370

Input promise: *"We reiterate our position since the time of the Jan Sangh to the abrogation of Article 370."*

Stage	What Happened	Result
1. Encode	Promise text → 384-D vector using MiniLM	Vector: [0.12, -0.33, ...]
2. Search	Compared against 524 gold vectors using cosine similarity	Best match found: 2014 record about Articl
3. Match	"BJP reiterates its stand on Article 370..." from 2014, label=1 (Partial), similarity=0.6136	Score 0.61 → Moderate Confidence tier
4. Semantic Output	Label 1 = 'Partial'. Score 0.61 → Moderate Confidence	Forecast: 'Moderate Confidence: Partial'
5. LLM Review	LLM knows Article 370 was ACTUALLY abrogated in Aug 2019. Disagrees with Partial label	LLM Verdict: 'Likely Fulfilled'
6. Discrepancy Flag	Semantic says Partial, LLM says Likely Fulfilled → discrepancy=true	Discrepancy note shown in output
7. Final Verdict	Semantic is primary → Final = 'Partially Fulfilled'. LLM note added as context.	final_verdict: Partially Fulfilled

Why does discrepancy happen here? The 2014 gold database labeled this as 'Partial' because in 2014, Article 370 had NOT yet been abrogated. But Article 370 WAS abrogated in 2019. The gold database reflects the state in 2014, not the final outcome. The LLM correctly identifies this gap — which is why the discrepancy flag is valuable for improving the data.

10. Architecture Summary

Complete system architecture from raw data to API response:

Layer	Component	Technology	Role
Data Ingestion	parser.py	pdfplumber, pandas	Extract text/tables from PDFs and CSVs
Labeling	labeler.py	SentenceTransformers + Cosine Similarity	Compare remark text → 0/1/2 label
Knowledge Base	gold_database.csv	CSV (524 rows)	Indexed historical promise-outcome pairs
Embedding Engine	SentenceTransformer	all-MiniLM-L6-v2 (384-D)	Convert text to meaning vectors
Semantic Search	util.cos_sim + torch.topk	PyTorch	Find top-3 most similar historical promises
Threshold Logic	resolve_final_verdict()	Python	Map similarity score to confidence tier
LLM Reviewer	llm_reviewer.py	Groq API (LLaMA 3.3 70B)	Political context review & discrepancy detection
API	main.py	FastAPI + uvicorn	REST endpoint exposing /predict and /health

11. API Endpoints

POST /predict

Accepts a JSON body with a promise text and returns the full analysis.

```
Request: {"text": "We will build 100 smart cities.", "use_llm": true} Response fields: final_verdict → Primary NLP output verdict_source → "semantic" or "semantic_only" discrepancy → true/false — LLM agrees or disagrees discrepancy_note → Explanation if discrepancy exists semantic_analysis → forecast, confidence score, reasoning llm_review → LLM verdict, reasoning, key_factors historical_evidence → Top 3 matched historical records
```

GET /health

```
Returns: { status, history_loaded, history_size, llm_ready }
```

12. Accuracy, Limitations & Future Scope

Strengths

- No dependency on expensive GPU — runs entirely on CPU
- No paid API required for core NLP — MiniLM is free and local
- Explainable output — every prediction comes with the actual historical evidence used
- LLM discrepancy flag helps identify where gold database labels may be wrong
- Sub-second inference speed (25ms for embedding, <100ms total without LLM)

Known Limitations

- Gold Database has label drift: 2014 labels reflect status-in-2014, not final-2019 outcomes (e.g., Article 370 marked Partial in 2014 data but was fulfilled in 2019)
- MiniLM was trained on general English — Indian political terminology and Hindi transliterations may reduce similarity scores
- Label distribution is imbalanced: ~59% label=2, which biases predictions toward 'Highly Likely'
- 2024 predictions are forecasts only — actual outcomes unknown until 2029

Future Scope

- Add Hindi/multilingual support using IndicBERT or MuRIL for regional language manifesto analysis
- Expand gold database to include state-level manifesto data (UP, Maharashtra, etc.)
- Add Named Entity Recognition (NER) to extract specific entities — schemes, amounts, deadlines — from promises
- Automatically update gold database after each election term using web scraping
- Add confidence calibration layer to fix the label imbalance bias

13. How to Run the Project

Step 1 — Set up environment

```
python -m venv venv venv\Scripts\activate pip install -r requirements.txt
```

Step 2 — Rebuild Gold Database (one-time)

```
python rebuild_gold.py # Output: data/processed/gold_database.csv (524 records)
```

Step 3 — Set Groq API Key (for LLM layer)

```
$env:GROQ_API_KEY = "gsk_your_key_here" # PowerShell # Free key from:
https://console.groq.com
```

Step 4 — Start the API Server

```
uvicorn main:app --reload # Server runs at http://localhost:8000
```

Step 5 — Test a Promise

```
curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d
{"text": "We will double farmers income by 2022.", "use_llm": true}
```

Step 6 — Run Batch Analysis of 2024 Manifesto

```
python semantic_checker.py # Output: data/output/2024_fact_check_report.csv
```